

# Master Practical Course: Edge Computing and the Internet of Things - Group BOM

Ivo Raimondi  
raimondiivo@gmail.com  
Technical University of Munich  
Munich, Germany

Inessa Kechek  
inessa.kechek@etu.sorbonne-  
universite.fr  
Technical University of Munich  
Munich, Germany

Rodrigo Forno  
rodrigo.felix@tum.de  
Technical University of Munich  
Munich, Germany



Figure 1: Our finished robot along with our team members of Ivo (left), Rodrigo (middle), Inessa (right).

## ACM Reference Format:

Ivo Raimondi, Inessa Kechek, and Rodrigo Forno. 2026. Master Practical Course: Edge Computing and the Internet of Things - Group BOM . In *Proceedings of Master Practical Course: Edge Computing and the Internet of Things (CM '26)*. ACM, New York, NY, USA, 9 pages.

## 1 Introduction

A robot management system is a system capable of connecting and managing a fleet of independent robots. These robots might be used together in a mission or separately, but managed centrally. This project set out to design, implement and evaluate a prototype of this system, using the base use-case of exploration of dangerous closed environments, such as a sewage system. Our objectives included

- (1) Video streaming from IoT to Edge (and further to the cloud)
- (2) Autonomous movement

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

CM '26, Munich, Germany

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

- (3) SLAM capabilities
- (4) Environment sensing

The challenges included striking the balance between energy autonomy and edge computing power, as well as architectural considerations for allowing the system to scale up well enough to handle thousands of robots. To tackle this, we designed both a prototype rover car as well as the processing and user interface services required.

## 2 Background and Motivation

### 2.1 Motivation: Why Build This Rover?

Unmanned Ground Vehicles (UGVs) are increasingly required for tasks such as infrastructure monitoring, particularly in underground or indoor environments where aerial alternatives are impractical and human inspection may be dangerous or unfeasible. These systems must combine high autonomy with energy efficiency, as they often operate under unstable communication conditions and for extended exploration periods.[5]. Current approaches primarily focus on optimizing individual robotic platforms for complex terrain and adversarial conditions. However, they rarely exploit the potential of the Internet of Things (IoT) within robotics and autonomous systems—such as improved decision-making, adaptability through

collaboration, and enhanced predictive capabilities via large-scale data aggregation [6]. Our project aimed to investigate the architecture required for realizing this task and exploring the trade-offs involved when needing to balance scale of data processing, autonomy, energy efficiency and architectural simplicity. A further discussion of related work comes later in this report.

## 2.2 Background Technologies

- **MQTT:** MQTT (Message Queuing Telemetry Transport) is a lightweight messaging protocol based on the publish/subscribe paradigm, realized with TCP/IP [1]. Instead of communicating directly with each other, clients connect to a central broker. Clients can publish messages to specific topics, while other clients subscribe to these topics to receive the corresponding messages. The broker handles message distribution, decoupling senders and receivers. The use in this practical project was to relay sensor data from the rovers to the cloud backend and receive commands from the cloud to the rover.
- **FreeRTOS:** FreeRTOS (Free Real-Time Operating System) is a lightweight real-time operating system designed for microcontrollers and embedded systems. It provides task scheduling, inter-task communication, and timing mechanisms, of which we mostly used task scheduling [2]. Tasks in FreeRTOS are executed according to a priority-based preemptive scheduler, ensuring that time-critical operations can be handled deterministically. This operating system was used in the ESP32.
- **SLAM (Simultaneous Localization and Mapping):** SLAM is a computational technique used to construct a map of an unknown environment while simultaneously tracking a robot's location within that space. In this project, a custom 2D SLAM engine was deployed on the edge (Raspberry Pi 5). It fuses internal odometry data—gathered from a single wheel encoder and an IMU gyroscope—to continuously estimate the robot's real-time pose  $(x, y, \theta)$ . Simultaneously, it processes exteroceptive distance readings from an array of four Time-of-Flight (ToF) sensors to incrementally build a probabilistic 2D occupancy grid map of the environment. [7]
- **WebRTC (Web Real-Time Communication):** WebRTC is an open-source protocol that enables direct, peer-to-peer communication of audio, video, and data across web browsers and devices with sub-second latency. It natively handles complex network routing challenges, such as NAT traversal, to establish direct connections. In this project, WebRTC was utilized to achieve real-time, low-latency video teleoperation. Raw video captured by the robot's onboard camera is compressed into an H.264 RTSP stream using FFmpeg. A local MediaMTX [8] server running on the Raspberry Pi translates this RTSP feed into a WebRTC stream. This architecture allows the React-based frontend UI to consume and display the live video natively within the browser, providing the operator with a highly responsive visual feed of the environment without the need for external browser plugins.

## 2.3 Related Work / Similar Projects

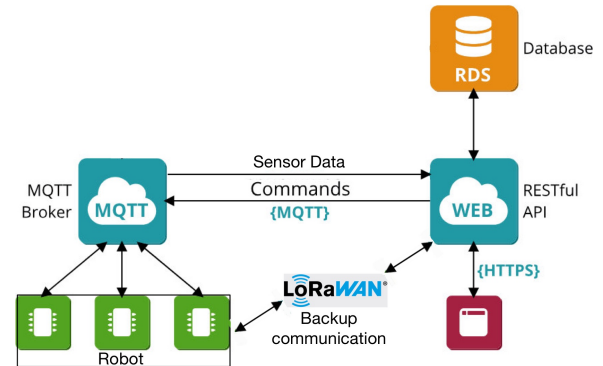
The application of edge computing for autonomous mobile robots has been widely explored to address the latency, energy, and connectivity constraints of traditional cloud-based systems. In the context of industrial environments, S. et al. [9] successfully demonstrated an autonomous rover equipped with a Jetson Nano for real-time safety compliance monitoring. Their architecture utilizes a hybrid edge-cloud approach, using Azure Cloud for high-level monitoring while maintaining real-time video analytics at the edge.

While this hybrid approach is highly effective in stable environments with reliable network infrastructure, it becomes vulnerable in unpredictable scenarios. Addressing this, Militano et al. [4] explored the broader Cloud-to-Edge-to-IoT continuum specifically for Search and Rescue (SAR) operations. They emphasize that in hazardous and disaster-stricken environments, relying on distant cloud servers introduces unacceptable latency and points of failure, making localized edge nodes critical for mission success.

## 3 Architecture and System Design

### 3.1 System Overview

The system can be decomposed onto three main components: the rover(s), the cloud processing service and the user interface service. The architectural diagram 2 gives a clear overview of this architecture.



**Figure 2: High-level system architecture consisting of rover, cloud processing, and user interface components.**

In essence, the robots should identify themselves with the cloud service and establish a resilient connection through MQTT. This newly available robot is then reachable to users, which can visualize sensor data, camera data and control the robot through the interface provided by a web service. Although a LoRaWAN backup connection was planned in the original architecture, the final implementation couldn't contain it in the interest of time.

There's a clear hierarchical aspect to this architecture, which was chosen due to the use case of these rovers: a company/organization would like to roll out their own services for running these missions. A similar architectural approach is used for more complex data scale, such as the case of BMW [10].

## 3.2 Software Architecture

**3.2.1 ESP32.** The ESP32 microcontroller runs a modular, real-time firmware built on the Arduino framework and managed by FreeRTOS. The project is structured using PlatformIO's library convention: each sensor or actuator driver resides in its own directory under `lib/`, exposing a minimal header that hides hardware-specific details from application code. The main application (`src/main.cpp`) instantiates every driver, spawns two FreeRTOS tasks, and delegates all runtime work to them; the Arduino `loop()` function remains empty.

Two tasks partition the firmware's responsibilities:

- **TaskReadAndSendSensorData** (priority 1) — periodically polls every sensor at a configurable interval (100 ms by default) and transmits the aggregated telemetry over UART to the Raspberry Pi 5.
- **TaskReceiveAndHandleCommands** (priority 3) — listens for incoming commands on the UART link and translates them into DriveMode updates. Its higher priority ensures that drive commands are handled promptly, even when a sensor read is in progress.

**Motor Control.** The rover's motor control is implemented on the ESP32 using an Arduino-based firmware architecture and a MotorShield abstraction layer. A dedicated DriveController class encapsulates the coordination of four DC motors (front-left, front-right, rear-left, rear-right) through a MotorShield abstraction layer. Speed control is implemented using PWM with an 8-bit resolution (0–255), where the duty cycle is applied symmetrically to all motors in forward and backward modes. Directional control is achieved by commanding each motor independently, allowing coordinated inversion of rotation for differential steering.

The motor control loop is executed via the `update()` method within the FreeRTOS task context, applying the current mode and speed settings to all motors. Some experiments with closed-loop feedback control were conducted, which can be explored in further detail under `autonomous_control` in the `docs-and-resources` repository [3]. Instead, a simpler constant acceleration pattern was used, as can be seen in figure 3. It's to be noted that the graph only contains 2 motors. This is a simplification done on the kinematic model used for the rover called *unicycle model*. This didn't noticeably affect the rover's performance.

**Sensor Wrapper Libraries.** To abstract hardware-specific register operations and third-party library internals, each sensor has a wrapper exposing standard `init()` and `read()` interfaces. A summary of the integrated sensors, their communication interfaces, and specific wrapper configurations is provided in Table 1.

**UART Communication Protocol.** The ESP32 and Raspberry Pi 5 communicate via a dedicated hardware UART (`Serial1`) at 115 200 baud. The protocol relies on line-delimited JSON, parsed via *ArduinoJson*, with each message terminated by a newline (`\n`).

**Telemetry (ESP32 → RPi):** Every 100 ms, the ESP32 transmits a timestamped JSON object (Listing 1) containing all sensor readings. To minimise heap allocations on the microcontroller, the payload is constructed via direct string concatenation. Missing measurements are serialised as `null` to differentiate them from valid zero readings.

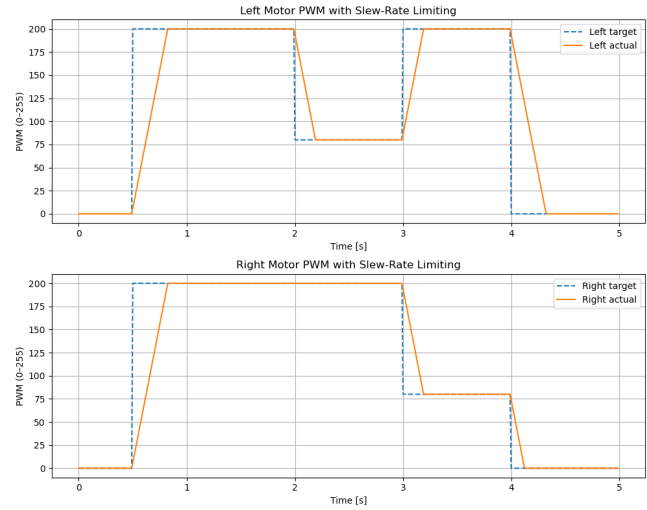


Figure 3: Constant acceleration of DC motors

Table 1: Summary of Sensor Wrapper Implementations

| Sensor       | Measurement                  | Interface (Addr)             | Key Configurations & Features                                                                                 |
|--------------|------------------------------|------------------------------|---------------------------------------------------------------------------------------------------------------|
| BME280       | Env. (Temp, Press, Hum, Alt) | I <sup>2</sup> C (0x76)      | C-compatible opaque API; internal NaN validation to prevent upstream errors.                                  |
| MPU6050      | IMU (3-axis Accel/Gyro)      | I <sup>2</sup> C             | Range: $\pm 8g$ , $\pm 500 \text{ deg/s}$ ; 5 Hz hardware low-pass filter to attenuate chassis vibrations.    |
| VL53L0X (x4) | Time-of-Flight Distance      | I <sup>2</sup> C (0x30–0x33) | Dynamic address reassignment via XSHUT GPIOs; continuous ranging mode with 500 ms timeout.                    |
| AS5600       | Magnetic Rotary Encoder      | I <sup>2</sup> C (0x36)      | 12-bit raw to float degree conversion; includes <code>tare()</code> function and magnet presence diagnostics. |

Listing 1: Example telemetry JSON frame.

```
{
  "device": "rover-esp32-001",
  "bme_temp_c": 23.45, "bme_press_hpa": 1012.30,
  "bme_hum_pct": 41.20, "bme_alt_m": 8,
  "imu": {
    "ax": 0.12, "ay": -0.03, "az": 9.81,
    "gx": 0.001, "gy": -0.002, "gz": 0.000,
    "temp_c": 25.10
  },
  "tof": {
    "d1": 245, "d2": 1023, "d3": 87, "d4": 530
  },
  "encoder": {
    "e1": 182.3
  },
  "timestamp_ms": 104200
}
```

**Commands (RPi → ESP32):** Incoming commands are read into a 256-byte buffer. The parsed "command" field maps to discrete modes (*forward*, *backward*, *left*, *right*, *stop*). As a safety measure, any unrecognised command immediately halts the rover.

**3.2.2 Raspberry Pi 5 (Edge Compute Node).** The Raspberry Pi 5 (RPi5), running the lightweight Raspberry Pi OS (Raspbian) Lite, serves as the primary edge computing layer of the robot. Its software architecture is containerized using **Docker Compose**, which ensures reproducible deployments and isolates three main background services:

- **Data Bridge and Orchestrator (uart2mqtt.py):** This Python-based service acts as the central nervous system on the edge. Upon initialization, it resolves its local network IP address and transmits it to the MQTT broker, ensuring the cloud backend knows how to directly route video traffic to the robot. It then continuously reads high-frequency sensor payloads from the ESP32 via the serial UART connection. It parses these JSON payloads, feeds them into the local SLAM engine, and subsequently publishes the processed telemetry, robot pose, and compressed map data to the cloud backend via MQTT. Conversely, it subscribes to the cloud command topics and forwards driving instructions back to the ESP32.
- **Video Streaming Pipeline (FFmpeg & MediaMTX):** To provide the operator with real-time visual feedback, the RPi5 runs a highly optimized video pipeline. FFmpeg captures raw frames directly from the USB camera interface (/dev/video0), hardware-encodes them to H.264 using ultra-low latency presets, and pushes an RTSP stream locally. A MediaMTX container receives this RTSP feed and dynamically serves it as a WebRTC stream.
- **Custom SLAM Engine:** A custom Simultaneous Localization and Mapping (SLAM) algorithm was developed in Python. The engine employs a probabilistic 2D occupancy grid mapping approach (at a 5cm resolution). It fuses data from the single wheel encoder and the IMU's Z-axis gyroscope for odometry. Because the robot relies on only one encoder, an in-place rotation would incorrectly be registered as forward or backward translation as the wheel spins. To solve this, a command-based turn detection system is used: when a rotation command is active, the engine completely ignores the encoder ticks and calculates angular displacement exclusively using the IMU. Distance readings from four Time-of-Flight (ToF) sensors are then integrated using Bresenham's line algorithm [16] to update the map. By utilizing integer-based mathematics (such as Bresenham's algorithm) instead of floating-point operations, and compressing the map grid using zlib and encoding it in Base64 before sending it to the cloud, the RPi5 efficiently handles these algorithms.

**3.2.3 Server.** The cloud infrastructure is designed to remotely manage a fleet of autonomous rovers. It is divided into a robust backend API and a responsive web-based frontend, both containerized for rapid deployment.

- **Backend Infrastructure:** The backend is a Node.js-based system designed to manage the data lifecycle of the rovers. Its architecture is divided into three primary functional pillars: real-time data ingestion, persistent storage management, and a RESTful API for external consumption. For data ingestion, it utilizes an MQTT ingestion service to maintain persistent connections with a central broker. It subscribes to a structured topic hierarchy (robots/ID) to receive telemetry, SLAM updates, and status reports from various rovers with specific ID. To ensure system performance under high data throughput, the backend utilizes a buffering strategy for database operations. The database used is TimescaleDB [11], a PostgreSQL variant adapted for timeseries data. At last, the

backend exposes a REST API built with Express to provide data to the frontend and generally offers the following:

- (1) Sensor data retrieval
  - (2) Downlink commands to the rover
  - (3) Mission oversight, such as status, rover connection status
  - (4) SLAM data retrieval
- **Frontend Application (React UI):** The operator interface is a Single Page Application (SPA) built with **React.js**, **TypeScript**, and **Vite**, styled using Tailwind CSS. The application architecture is built around three core workflows:
    - (1) **Fleet Management:** A Rover Selection portal that polls the backend to display available robots, their online status, and the dynamically assigned local IP addresses (originally broadcasted by the RPi5) required for video streaming.
    - (2) **Mission Control Dashboard:** The primary teleoperation hub. This interface aggregates real-time sensor telemetry, visualizes the live 2D SLAM map, and provides a control pad for sending manual driving commands. It also handles mission lifecycle management (starting, describe, and concluding missions with operator notes).
    - (3) **Historical Data:** A Missions portal that retrieves and displays past mission metrics, final map snapshots, and environmental data variations.

To ensure optimal performance, the frontend employs a dual-communication strategy: standard interactions use HTTP/REST to the backend, while the live video feed uses a direct, peer-to-peer WebRTC connection to the robot.

### 3.3 Network and Communication Architecture

**3.3.1 MQTT.** MQTT provides configurable resilience through three Quality of Service (QoS) levels as well as a "Last Will and Testament" (LWT) feature, ensuring the backend is notified of connection loss. Additionally, the protocol's persistent sessions and automatic reconnection logic allow the system to recover state and buffer messages during transient network failures.

To manage a fleet of multiple rovers, the system implements a structured and hierarchical topic namespace. The backend subscribes to a multi-level wildcard pattern (robots/ID) to ingest all incoming traffic, which is then parsed using the following convention:

- robots/{robot\_id}/telemetry: Used for high-frequency sensor streams (IMU, environmental data).
- robots/{robot\_id}/status: Monitors the heartbeat and connectivity state (LWT) of each unit.
- robots/{robot\_id}/slam/{pose|map}: Dedicated channels for spatial awareness data, including coordinate frames and occupancy grids.
- robots/{robot\_id}/command: A downlink channel used by the API to publish remote control instructions back to the rovers.

In a production scenario, rovers would authenticate themselves using a X509 certificate.

**3.3.2 WebRTC.** For live visual feedback during manual teleoperation, the system utilizes WebRTC. Instead of routing the heavy video feed through the cloud backend—which would introduce significant latency and bandwidth costs—the React frontend utilizes



the robot's dynamically assigned local IP (fetched from the backend's database) to establish a direct, peer-to-peer connection with the Raspberry Pi's MediaMTX server.

The primary justification for WebRTC is its underlying reliance on UDP (User Datagram Protocol). In remote driving scenarios, "glass-to-glass" latency<sup>1</sup> is the most critical metric. WebRTC prioritizes speed over perfect packet delivery; if a video frame is corrupted or lost in transit, the protocol simply drops it and renders the next available frame, avoiding the head-of-line blocking typical of TCP. This acceptable tradeoff ensures the operator experiences sub-second latency, which is strictly necessary for safe and responsive obstacle avoidance.

**3.3.3 HTTP & REST (Command and Control).** While video requires speed, mission-critical commands and system state management require absolute reliability. All non-video interactions between the frontend and the cloud backend—such as fetching mission logs, updating robot statuses, and sending manual drive commands (`/api/drive/:id/:cmd`)—are handled via standard HTTP REST APIs.

Because HTTP operates over TCP (Transmission Control Protocol), it guarantees sequential packet delivery and error checking. This is strictly required for robot control and database operations. For example, if an operator issues a "STOP" command, or submits a final comment to conclude a mission, losing that data packet in transit is unacceptable. The HTTP protocol ensures these requests securely reach the backend, which then reliably bridges the drive commands to the MQTT broker for execution by the edge nodes.

### 3.4 Design Justifications

**3.4.1 Two-Tier Edge Architecture.** The rover's compute stack is intentionally split: a microcontroller (ESP32) handles hard real-time sensor acquisition and motor control, while a single-board computer (Raspberry Pi 5) manages higher-level navigation and network connectivity. This separation ensures **determinism** (FreeRTOS guarantees cycle-accurate timing for PWM and I2C polling) and **isolation** (a Pi kernel fault will not disrupt the low-level control loop, allowing the ESP32 to safely halt). Furthermore, this enables **fleet scalability**; the ESP32 firmware remains identical across the fleet, while complex, easily updatable coordination logic resides exclusively on the Pi.

**3.4.2 Internal Link: UART vs. Wireless/Bus.** Communication between the ESP32 and Raspberry Pi utilises a physical hardware UART. Unlike Wi-Fi or Bluetooth, UART introduces zero pairing overhead, providing the deterministic, sub-millisecond latency required for the 100 ms control period. Compared to SPI, UART simplifies the physical wiring harness (requiring only TX/RX) and operates in full-duplex, allowing simultaneous telemetry and command flow without a restrictive master-slave polling architecture.

**3.4.3 Telemetry Format: JSON Trade-offs.** All inter-processor messages are line-delimited JSON. While less bandwidth-efficient

than packed binary protocols, JSON was selected for its distinct practical advantages:

- **Self-describing fields:** Adding new sensors only requires appending a key-value pair.
- **Fleet heterogeneity:** A single parser handles mixed-fleet variants, as each rover simply advertises its available hardware.

**3.4.4 Edge-First Computation Strategy.** One of the main features of this project is confining real-time computation to the edge, rather than offloading to a cloud server. This is justified by:

- **Latency bounds:** Closed-loop obstacle avoidance requires millisecond reaction times. Cloud round-trips introduce variable latency and packet loss risks.
- **Bandwidth scaling:** Streaming raw telemetry from  $N$  robots scales linearly, overwhelming network infrastructure. Processing locally and transmitting only high-level events dramatically reduces aggregate bandwidth.

Centralised cloud servers are therefore strictly reserved for asynchronous tasks that benefit from global context, such as map merging or fleet task allocation.

## 4 Sensors, Components and Hardware

### 4.1 Hardware Architecture

**4.1.1 ESP32.** As discussed earlier, the ESP32 serves as the rover's low-level controller. Powered at 3.3 V, it simultaneously drives four motors and polls seven I<sup>2</sup>C sensors.

The rover's environmental and spatial awareness is driven by a 3.3 V sensor suite, detailed in Table 2.

Table 2: Sensor Hardware Specifications

| Component    | Measurement      | Range / Resolution      | Accuracy          |
|--------------|------------------|-------------------------|-------------------|
| BME280       | Temp & Pressure  | −40–85 °C, 300–1100 hPa | ±1.0 °C, ±1.0 hPa |
| MPU6050      | Accel & Gyro     | ±8 g, ±500 °/s          | 5 Hz DLFP         |
| VL53L0X (x4) | Time-of-Flight   | 30–2000 mm              | ±3%               |
| AS5600       | Magnetic Encoder | 12-bit (4096 pos/rev)   | ±1 °              |

Functionally, the IMU's hardware low-pass filter (DLFP) is crucial for attenuating severe chassis vibrations generated by the motors, while its wide accelerometer range easily captures high-impact terrain shocks. The four Time-of-Flight modules are arrayed to provide a 180° proximity sweep for collision avoidance. Ambient environment logging is handled by the BME280, and the magnetic encoder facilitates odometry tracking on the drivetrain.

**4.1.2 Raspberry Pi 5.** The RPi5 functions as the central compute hub in the edge architecture, designed to handle computationally expensive tasks like SLAM that exceed the ESP32's capabilities. Its physical integration relies on three main interfaces:

- **Camera Integration:** Visual telemetry is captured using a standard USB webcam. Utilizing USB instead of a proprietary CSI ribbon cable allowed for greater mounting flexibility on the chassis and simplified containerised driver mapping. The camera is recognised via the Video4Linux2 (V4L2) framework at `/dev/video0`, streaming MJPEG at 640x480 pixels and 30 fps.

<sup>1</sup>"Glass-to-glass" latency denotes the elapsed time between photons striking the camera sensor and the corresponding pixels illuminating on the operator's display. The term is widely used in real-time video and broadcast systems to describe total visual pipeline delay.

- **Serial Interfacing (UART):** The RPi5 utilises its primary hardware serial port (`/dev/ttyAMA0`) to maintain a high-speed, reliable data link with the ESP32.
- **Power Reception:** To support the simultaneous processing of SLAM algorithms and hardware video encoding, the board requires a robust 5 V power supply capable of delivering up to 5 A. This is injected directly via the GPIO header from the Logic Subsystem power rail (see Section 4.1.4).

**4.1.3 Motors and Motor Drive.** The rover is driven by four DC motors (*TT motors*) operating between 6–12 V with a 1:48 gear reduction ratio, providing sufficient torque for independent mobility. These are controlled by a dual H-bridge 4-channel motor driver board, which receives PWM signals (0–5 V logic). The H-bridges allow the system to reverse current direction and modulate voltage, granting full control over the rotational direction and intensity of the wheels.

**4.1.4 Power Architecture.** To ensure system stability and isolate sensitive microelectronics from inductive noise and voltage drops caused by motor stall currents, the rover employs a physically separated dual-battery power architecture. Power is supplied by two identical 7.4 V, 3300 mAh LiPo batteries:

- The first battery (Logic Subsystem) strictly powers the compute and sensor stack. Its 7.4 V output is regulated via a 5 V step-down (buck) converter, creating a stable power rail for the Raspberry Pi 5, USB camera, and the ESP32 with its sensors.
- The second battery (Drive Subsystem) is dedicated to actuation. It connects directly to the motor driver shield, providing 7.4 V to the four DC motors.

**4.1.5 CAD.** To house all of the components, a few harnesses were designed and 3D-printed using PLA. These include:

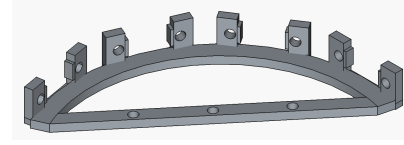
- a chassis that houses the motors, the motor driver board as well as the ESP32 mounted on a protoboard
- mounts for the magnetic wheel velocity encoders
- a array mount for the Time-of-Flight sensors
- an upper mount for the camera as well as the Raspberry PI 5

A few pictures of the designed partes are included in Figure 4. Components were connected using heat inserts for M3 screws. For non-load-bearing components where threaded inserts were impractical, high-adhesion tape was utilised for rapid prototyping.

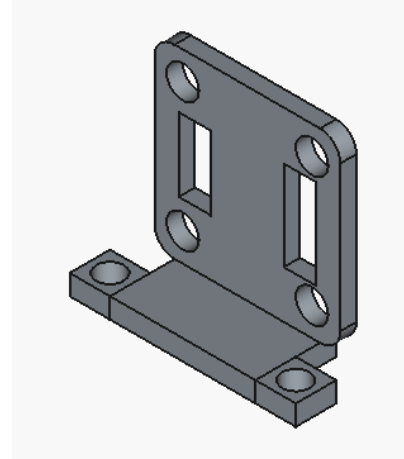
## 4.2 Wiring & Setup

The physical integration of the rover's components requires careful management of shared buses, logic levels, and GPIO limits across the chassis. The complete GPIO allocation for the ESP32 is summarised in Table 3.

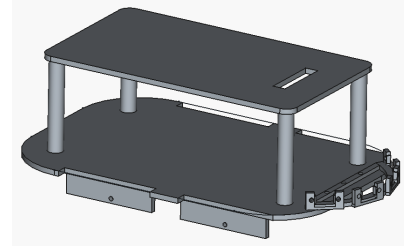
**I<sup>2</sup>C Bus and Sensor Addressing.** All seven sensors are wired to a single I<sup>2</sup>C bus (GPIO 21/22). A significant wiring challenge arises from the four VL53L0X modules, which natively share the same factory address (0x29). To avoid bus collisions without introducing an I<sup>2</sup>C multiplexer, four dedicated XSHUT lines are wired to the ESP32. During setup, the firmware holds all four XSHUT pins low (hardware reset). It then sequentially pulls each pin high, waking



(a) 4-sensor ToF Array Mount



(b) Wheel velocity encoder Mount



(c) Chassis and Upper Plate

Figure 4: Custom 3D-printed hardware components

Table 3: ESP32 GPIO Pin Allocation and Wiring

| Function                     | GPIO              | Wiring / Setup Notes              |
|------------------------------|-------------------|-----------------------------------|
| I <sup>2</sup> C (SDA / SCL) | 21 / 22           | Shared bus (100 kHz), 3.3 V logic |
| UART (TX / RX)               | 17 / 16           | Direct to RPi UART                |
| ToF XSHUT (1–4)              | 4, 27, 32, 33     | Active-low hardware reset lines   |
| Motor Latch / Clk / Data     | 5 / 18 / 23       | To motor shield shift register    |
| Motor Enable                 | 19                | Active-low output enable          |
| Motor PWM (FL/FR/RL/RR)      | 25 / 26 / 14 / 13 | Dedicated hardware timer channels |

the sensors one by one and dynamically assigning them unique addresses (0x30–0x33) via software.

Furthermore, the AS5600 encoder possesses a fixed 0x36 address, limiting the current design to a single encoder on the bus. Expanding to multi-wheel odometry would require adding an I<sup>2</sup>C multiplexer.

**Logic Levels and Interconnects.** Because both the ESP32 and the Raspberry Pi 5 operate at 3.3 V logic levels, the UART TX/RX lines are wired directly between the two boards without the need

for an intermediate level shifter. This reduces point-to-point wiring complexity.

## 5 Evaluation and Measurements

### 5.1 Energy Consumption of the System

The rover's energy profile was measured by evaluating through data-sheets the current draw of individual components. Each 7.4 V, 3300 mAh LiPo battery provides a total capacity of approximately 24.4 Wh.

**Logic Subsystem (Battery A) Measurements.** The logic rail operates through a 5 V step-down converter with an estimated 90% efficiency. It was estimated that there is a continuous draw of  $\sim 6$  W ( $\sim 1200$  mA) from the Raspberry Pi 5 with the CPU active and Wi-Fi enabled. The USB camera streaming video consumed  $\sim 1.5$  W ( $\sim 300$  mA), while the ESP32 and sensor suite drew a combined  $\sim 0.32$  W. The total measured logic load is 7.82 W ( $\sim 1.6$  A).

**Drive Subsystem (Battery B) Measurements.** Under normal driving loads, the four DC gear-motors connected directly to the unregulated 7.4 V battery draw approximately 1.6 A (11.84 W). Factoring in the minor logic overhead ( $\sim 0.25$  W) required for the motor shield, the total measured drive load is 12.1 W.

**Table 4: Measured Power Budget and Subsystem Loads**

| Subsystem      | Key Components                | Voltage | Nominal Power |
|----------------|-------------------------------|---------|---------------|
| Logic (Bat. A) | RPi 5, Camera, ESP32, Sensors | 5.0 V   | 7.82 W        |
| Drive (Bat. B) | 4× DC Motors, Motor Shield    | 7.4 V   | 12.10 W       |

**Calculated Autonomy.** The overall system autonomy is bottlenecked by the drive subsystem. Accounting for the 5 V step-down converter's 90% efficiency, the logic subsystem's effective power draw from the 24.4 Wh battery increases to 8.7 W, yielding a theoretical runtime of 2.8 hours. Conversely, the 12.1 W direct draw on the drive battery depletes its 24.4 Wh capacity in exactly 2.0 hours. Thus, the rover guarantees two hours of fully autonomous operation per charge cycle.

### 5.2 Data Transmission Rate

To evaluate internal network efficiency, the throughput of the ESP32-to-RPi5 UART link was measured. Operating at 115 200 baud the theoretical maximum channel capacity is 11.52 KB/s.

During autonomous operation, the ESP32 generates a comprehensive telemetry frame containing all sensor data and system states every 100 ms (10 Hz). The average line-delimited JSON payload measures 350 bytes per frame, resulting in a continuous telemetry data rate of  $\sim 3.5$  KB/s.

Consequently, telemetry traffic consumes approximately 30% of the available UART bandwidth. This leaves over a 70% margin available for the Raspberry Pi to send high-frequency drive commands down to the ESP32. This low utilization proves that the JSON-over-UART architecture easily meets the system's real-time communication constraints without introducing queueing delays.

### 5.3 Video Latency and WebRTC

While exact millisecond "glass-to-glass" latency was not quantitatively benchmarked using high-speed frame analysis, empirical testing strongly validated the WebRTC architectural choice. Real-world operator tests—such as performing sudden movements directly in front of the rover's camera—resulted in near-instantaneous visual updates on the React frontend.

This visually confirmed that the sub-second latency target was achieved. The absence of noticeable buffering, frame tearing, or lag during manual teleoperation proved that bypassing the cloud backend and utilizing a local, UDP-based WebRTC connection was highly effective. The operator was able to drive the robot safely relying solely on this visual feedback, fulfilling the primary requirement for real-time remote control.

In terms of teleoperation usability, while the streaming performance was flawless, the physical placement of the camera presented a minor limitation. Due to space constraints on the current prototype, the camera was mounted towards the rear of the rover. Consequently, the lower portion of the camera's field of view is partially occluded by the Raspberry Pi 5 and the front chassis structure. While operators were still able to navigate the environment successfully, this occlusion slightly reduced ground-plane visibility for close-proximity obstacles. A future hardware redesign that relocates the camera to the front edge—or elevates it on a dedicated mount—would maximize the effective field of view and make remote driving significantly easier.

### 5.4 Qualitative Measurements and System Integration

Beyond strict power metrics, the overall success of the architecture was evaluated qualitatively through end-to-end integration testing. The system successfully demonstrated cohesive operation across the edge, cloud, and frontend layers:

- **SLAM and Spatial Awareness:** The custom 2D SLAM engine was evaluated based on dynamic map generation rather than strict quantitative accuracy against a ground-truth system. As the robot navigated and rotated, the React frontend successfully visualized the live occupancy grid, continuously drawing walls and environmental boundaries. While the generated map exhibited minor drift and was not geometrically perfect—a known and expected limitation of relying on a single wheel encoder and IMU for odometry—it proved functional for real-time spatial awareness.
- **Real-Time Telemetry:** The end-to-end sensor data pipeline operated seamlessly. Environmental metrics (temperature, humidity) and IMU readings captured by the ESP32 were bridged through the RPi5, routed via the cloud MQTT broker, and dynamically updated on the frontend dashboard without perceptible delay.
- **Mission Persistence and Lifecycle:** The system's state management and database architecture functioned entirely as designed. Operators could seamlessly initiate a mission, record live telemetry and map data, append post-mission operator notes, and securely save the session. The successful retrieval of these sessions from the historical Missions portal

confirmed that the backend provided robust and reliable data persistence.

## 6 Conclusion and Future Work

### 6.1 Future Work

While the current prototype successfully achieved its remote tele-operation goals, the ideal vision for this platform encompasses a transition from manual control to semi-autonomous navigation. To realize this vision, future iterations of the system will focus on two primary domains:

- **Enhanced SLAM and Odometry:** As noted in the system evaluation, the custom 2D SLAM engine is currently constrained by sparse exteroceptive data (four ToF sensors located at the front of the robot) and basic odometry (IMU gyroscope and encoder). The current prototype utilizes a single wheel encoder, which prevents the system from accurately calculating differential rotation. While the ideal solution is to implement dual-channel magnetic encoders on both the left and right wheels to enable differential odometry, this was not possible in the current iteration as the ESP32's I2C bus was already at maximum capacity supporting the sensor array. Future hardware revisions will utilize an I2C multiplexer or a secondary microcontroller to accommodate these encoders. Additionally, replacing the current ToF array with a 360-degree 2D LiDAR would provide high-density environment data, making the mapping and navigation process significantly easier and more reliable.
- **Intelligent Inference:** Future iterations of the platform will focus on transitioning from a reactive data store to an active analytical engine by integrating **Apache Spark** for distributed stream processing. Using Spark along with Edge pre-processing will allow rovers to perform local data reduction, ensuring only useful features get passed along to Spark and speeding up inference. Use cases include: "dangerous" score using sensor data for mapping zones that requires maintenance, highly unstable environment whenever LiDAR readings start given anomalous readings.

### 6.2 Conclusion and IoT Learnings

Developing this rover prototype provided profound practical insights into the complexities of Internet of Things architectures and Edge computing. A primary takeaway was the critical necessity of the edge-compute paradigm for autonomous systems: by localising the computationally expensive SLAM engine and video encoding on the Raspberry Pi 5, the system successfully circumvented the severe latency, packet loss, and bandwidth bottlenecks inherent to cloud-offloading. Furthermore, the project underscored that there is no universal communication protocol in IoT; robust architectures require a hybrid approach tailored to data needs. This was demonstrated by our strategic use of WebRTC (UDP) for loss-tolerant, low-latency video streaming, contrasted with MQTT and HTTP (TCP) for mission-critical, guaranteed command delivery. Finally, the physical integration phase highlighted the tight coupling between hardware constraints and software design. Challenges such as resolving I<sup>2</sup>C address collisions without a hardware multiplexer,

adapting SLAM odometry for a single-encoder drivetrain, and managing a physically isolated dual-battery power budget reinforced that successful IoT deployment requires balancing computational ambition with strict physical, electrical, and network constraints.



## A Reproducibility Appendix

The complete source code, hardware schematics, and deployment configurations for this project are hosted on GitHub under the **TUM Edge IoT** organization [15]. The project is modularized into four primary repositories to ensure a clean separation of concerns:

- **Sensor Firmware (/sensor-firmware):** Contains the C++/FreeRTOS source code for the ESP32, including motor control and sensor wrappers [14].
- **Edge Computing Layer (/rpi5-on-board):** Contains the Dockerized software stack for the Raspberry Pi 5, including the SLAM engine and video pipeline [13].
- **Cloud Infrastructure (/cloud-app):** Contains the React.js frontend, the backend API responsible for mission management, and the deployment of the database and the mqtt broker [12].
- **Project Documentation (/docs-and-resources):** Serves as the central repository for high-level architecture and guides.

Detailed build instructions, environment variable requirements, and hardware assembly guides are provided in the README files of each respective repository.

## B SLAM Implementation Details

The custom SLAM engine developed for this project utilizes a probabilistic 2D occupancy grid approach, optimized for the Raspberry Pi 5's edge computing capabilities. The following sections summarize the core logic; the full technical documentation and source code are available in the project's repository [13].

### B.1 Probabilistic Occupancy Grid

To account for sensor noise from the Time-of-Flight (ToF) sensors, the map is not binary. Instead, each  $5\text{cm} \times 5\text{cm}$  cell contains a confidence value ranging from -100 (definitely free) to +100 (definitely occupied). The system uses an asymmetric update rule:

- **Obstacle Detection:** When a sensor reports a hit, the corresponding cell increases by +10. This ensures obstacles appear rapidly (within 1-2 readings).
- **Free Space:** Cells along the sensor ray are decremented by -1. This slower clearing rate prevents the map from "flickering" due to transient sensor errors.

### B.2 Bresenham's Line Algorithm

Efficient ray tracing is achieved using Bresenham's line algorithm, which relies exclusively on integer arithmetic to find all grid cells traversed by a ToF beam. This allows the RPi5 to update hundreds of cells per second without the overhead of floating-point calculations.

### B.3 Odometry and Command-Based Turn Detection

Due to the hardware limitation of having only a single wheel encoder, traditional differential odometry was impossible. The system instead uses a hybrid logic:

- (1) **Translation:** While moving straight, distance is calculated via the single magnetic encoder.
- (2) **Rotation:** Changes in heading ( $\theta$ ) are derived exclusively from the gyroscope.

- (3) **Turn Logic:** To prevent rotation being misidentified as translation, the SLAM engine intercepts "left" and "right" driving commands. When a turn is active, encoder input is ignored for position tracking ( $x, y$ ), and only the gyroscope updates the heading.

## B.4 Map Compression and MQTT Transmission

To maintain low-latency updates over the network, the  $400 \times 400$  grid (160 KB raw) is processed before transmission:

- (1) The probabilistic values are converted to a 1-byte display format.
- (2) The resulting buffer is compressed using zlib, typically achieving a 97% reduction in size (2–5 KB).
- (3) The compressed binary is Base64 encoded and published to the robots/{id}/slam/map MQTT topic.

## References

- [1] Andrew Banks, Ed Briggs, Ken Borgendale, and Rahul Gupta. 2019. MQTT Version 5.0. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>. OASIS Standard, Approved 7 March 2019.
- [2] FreeRTOS Community. 2024. Chapter 1: Multitasking in Small Embedded Systems. <https://github.com/FreeRTOS/FreeRTOS-Kernel-Book/blob/main/ch01.md#11-multitasking-in-small-embedded-systems>. Online; accessed September 8, 2026.
- [3] TUM Edge IoT. 2026. Documentation and resources for the project. <https://github.com/TUM-Edge-IoT/docs-and-resources>
- [4] Leonardo Militano, Adriana Arteaga, Giovanni Toffetti, and Nathalie Mitton. 2023. The Cloud-to-Edge-to-IoT Continuum as an Enabler for Search and Rescue Operations. *Future Internet* 15, 2 (2023), 55. doi:10.3390/fi15020055
- [5] Md. Abdur Rahim, Mohammad Rokonzaman, Ahmad Abu Alqumsan, Adetokunbo Arogbonlo, Md. Zohirul Islam, Hieu Trinh, and Md Shofiqul Islam. 2025. An intelligent and secure internet of robotic things: A review and conceptual framework. *Internet of Things* 33 (2025), 101684. doi:10.1016/j.iot.2025.101684
- [6] Pierluigi Rea and Erika Ottaviano. 2018. Design and development of an Inspection Robotic System for indoor applications. *Robotics and Computer-Integrated Manufacturing* 49 (2018), 143–151. doi:10.1016/j.rcim.2017.06.005
- [7] Bench Robotics. 2023. Differential Drive Robot Odometry Simulation in Python. <https://benchrobotics.com/concepts/differential-drive-robot-odometry-simulation-in-python/> Accessed: December 2026.
- [8] Alessandro Ros. 2026. *MediaMTX: Real-time media server and proxy*. <https://github.com/bluenviron/mediamtx> Accessed: November 2026.
- [9] K S S., K. Devi R., A. P., and D. A. Geethan. 2024. Automated Safety Compliance Monitoring in Industrial Environments with Autonomous Rover. In *2024 International Conference on Electrical Electronics and Computing Technologies (ICEECT)*. 1–7. doi:10.1109/ICEECT61758.2024.10739308
- [10] Rahul Shaurya, Junjie Tang, Patrick Lanners, and Simon Kern. 2022. BMW Cloud Data Hub: A Reference Implementation of the Modern Data Architecture on AWS. <https://aws.amazon.com/de/blogs/industries/bmw-cloud-data-hub-a-reference-implementation-of-the-modern-data-architecture-on-aws/>. AWS for Industries blog (06 July 2022)..
- [11] Attila Toth. 2023. *Time-Series Database: An Explainer*. Tiger Data. <https://www.tigerdata.com/blog/time-series-database-an-explainer> Accessed: 2025-05-22.
- [12] TUM Edge IoT Team. 2026. Cloud App: React UI and Backend Infrastructure. <https://github.com/TUM-Edge-IoT/cloud-app/>
- [13] TUM Edge IoT Team. 2026. RPi5 On-Board: SLAM and Video Pipeline. <https://github.com/TUM-Edge-IoT/rpi5-on-board/>
- [14] TUM Edge IoT Team. 2026. Sensor Firmware: ESP32 FreeRTOS Implementation. <https://github.com/TUM-Edge-IoT/sensor-firmware>
- [15] TUM Edge IoT Team. 2026. TUM Edge IoT: Autonomous Rover Project Organization. <https://github.com/TUM-Edge-IoT/Main-project-hub>.
- [16] Wikipedia contributors. 2026. Bresenham's line algorithm — Wikipedia, The Free Encyclopedia. [https://en.wikipedia.org/wiki/Bresenham%27s\\_line\\_algorithm](https://en.wikipedia.org/wiki/Bresenham%27s_line_algorithm) Accessed: January 2026.